

# VTU COMPUTER ORGANIZATION & ARCHITECTURE

## MODULE 2

*Addressing Modes, Assembly Language, I/O, Stacks, Subroutines & Additional Instructions*

| Topic                      | Description                                                            |
|----------------------------|------------------------------------------------------------------------|
| 1. Addressing Modes        | Register, Absolute, Immediate, Indirect, Index, Relative, Auto Inc/Dec |
| 2. Assembly Language       | Assembler directives, statement format, assembler functions            |
| 3. Basic I/O Operations    | Keyboard/display interface, DATAIN/DATAOUT, character programs         |
| 4. Stacks & Queues         | Stack operations (PUSH/POP), Queue, Stack vs Queue comparison          |
| 5. Subroutines             | Call/Return, nesting, parameter passing, stack-based linkage           |
| 6. Additional Instructions | Logical/Arithmetic shifts, Rotate instructions, BCD packing            |
| 7. Solved Problems         | 8 exam-style problems with complete solutions                          |
| 8. Quick Revision          | All key formulas, tables, and facts for last-minute study              |

**vtu-easy.com**

*Your complete VTU study companion*

---

## TABLE OF CONTENTS

---

- [1. Addressing Modes](#)
  - [1.1 Register Mode](#)
  - [1.2 Absolute \(Direct\) Mode](#)
  - [1.3 Immediate Mode](#)
  - [1.4 Indirect Mode](#)
  - [1.5 Index Mode](#)
  - [1.6 Base with Index Mode](#)
  - [1.7 Base with Index & Offset Mode](#)
  - [1.8 Relative Mode](#)
  - [1.9 Auto Increment Mode](#)
  - [1.10 Auto Decrement Mode](#)
- [2. Program: Add N Numbers \(Indirect Mode\)](#)
- [3. Program: Add N Numbers \(Auto Increment Mode\)](#)
- [4. Memory Mapped I/O vs I/O Mapped I/O](#)
- [5. Source Program vs Object Program](#)
- [6. Assembler Directives](#)
- [7. General Format of an Assembly Statement](#)
- [8. Assembler & Its Functions](#)
- [9. Two Pass Assembler](#)
- [10. Loader & Debugger Programs](#)
- [11. Basic Input/Output Operations](#)
- [12. Program: Read a Line & Display It](#)
- [13. Stack – Definition & Operations \(PUSH & POP\)](#)
- [14. Safe PUSH & POP Operations](#)
- [15. Reverse Polish Notation \(RPN\)](#)
- [16. Queue – Definition & Operations](#)
- [17. Stack vs Queue Comparison](#)
- [18. Subroutines – Call & Return](#)
- [19. Subroutine Nesting & Processor Stack](#)
- [20. Parameter Passing](#)
- [21. Logical Shift Instructions](#)
- [22. Program: Pack Two BCD Digits](#)
- [23. Rotate Instructions](#)
- [24. Solved Problems \(8 Problems\)](#)
- [25. Quick Revision Summary](#)

# 1. Addressing Modes

**Definition:** The different ways in which the location of an operand is specified in an instruction are referred to as **Addressing Modes**. The location is called the **Effective Address (EA)** of the operand.

An addressing mode tells the processor WHERE to find the operand — it may be in a register, directly given in the instruction, stored in memory, or computed from a combination of values.

## Computer Organization and Architecture Module 2 –Addressing modes

**Q. Define addressing mode. Explain any four basic addressing modes with syntax and examples.**

ADDRESSING MODES

- The different ways in which the location of an operand is specified in an instruction are referred to as **Addressing Modes** (Table 2.1).

**Table 2.1** Generic addressing modes

| Name                       | Assembler syntax                    | Addressing function                            |
|----------------------------|-------------------------------------|------------------------------------------------|
| Immediate                  | #Value                              | Operand = Value                                |
| Register                   | R <sub>i</sub>                      | EA = R <sub>i</sub>                            |
| Absolute (Direct)          | LOC                                 | EA = LOC                                       |
| Indirect                   | (R <sub>i</sub> )<br>(LOC)          | EA = [R <sub>i</sub> ]<br>EA = [LOC]           |
| Index                      | X(R <sub>i</sub> )                  | EA = [R <sub>i</sub> ] + X                     |
| Base with index            | (R <sub>i</sub> , R <sub>j</sub> )  | EA = [R <sub>i</sub> ] + [R <sub>j</sub> ]     |
| Base with index and offset | X(R <sub>i</sub> , R <sub>j</sub> ) | EA = [R <sub>i</sub> ] + [R <sub>j</sub> ] + X |
| Relative                   | X(PC)                               | EA = [PC] + X                                  |

Table 2.1 — Generic Addressing Modes Summary

*VTU Tip: EA = Effective Address = actual location of the operand. This is a key concept tested heavily in VTU exams.*

## 1.1 Register Mode

**Syntax:** R<sub>i</sub>

**EA:** EA = R<sub>i</sub>

- The operand IS the contents of a register. No memory access needed.
- The name (address) of the register is given explicitly in the instruction.
- Registers act as temporary storage locations where data is accessed very fast.
- Advantage:** Fastest mode — no memory access required.
- Use:** Temporary computations, intermediate results.

**Example:**

```
Move R1, R2 ; Copy contents of register R1 into register R2
```

Here, R1 is the source operand (register mode). The processor reads R1 directly without touching memory.

## 1.2 Absolute (Direct) Mode

**Syntax:** LOC

**EA:** EA = LOC

- The operand is in a MEMORY location.
- The address of the memory location is given EXPLICITLY (directly) in the instruction.
- The absolute mode can represent GLOBAL VARIABLES in a program.
- Advantage:** Simple and direct — good for global/static variables.
- Disadvantage:** Requires the full address to be embedded in instruction — makes instructions longer.

**Example:**

```
Move LOC, R2 ; Copy contents of memory-location LOC into register R2
```

The instruction encodes the full address LOC. The processor sends this address to memory and reads the value there.

### 1.3 Immediate Mode

**Syntax:** #Value**EA:** Operand = Value (no EA – value is the operand itself)

- The operand itself is GIVEN DIRECTLY in the instruction (not an address).
- The # symbol indicates immediate mode — the value follows directly.
- **Use:** Only used to specify the value of a SOURCE operand. Cannot be a destination.
- **Advantage:** No extra memory access needed — operand is part of instruction.
- **Limitation:** Cannot modify memory directly; value is fixed at assembly time.

**Example:**

```
Move #200, R0 ; Place the value 200 directly into register R0
```

The value 200 is embedded directly in the instruction encoding. No memory lookup is needed for the operand.

### 1.4 Indirect Mode

**Syntax:** (Ri) or (LOC)**EA:** EA = [Ri] or EA = [LOC]

- The EA of the operand is the CONTENTS of a register or memory location — not the register itself.

**Pointer.**

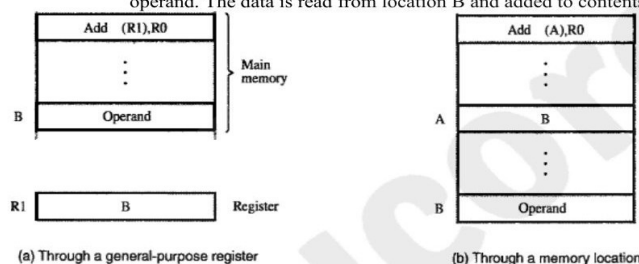
- Indirection is denoted by placing parentheses around the register/location name.
- **Register indirect:** EA = [Ri] — the register holds the address; one memory access for operand.
- **Memory indirect:** EA = [[LOC]] — two memory accesses (first to get address, then to get operand).

**Example:**

```
Add (R1), R0 ; Register R1 holds address B. Read value at B, add to R0.
Add (A), R0 ; Memory location A holds address B. Read [A]=B, then read [B]=operand.
```

→ new address given in the instruction.

E.g: *Add (R1),R0* ;The operand is in memory. Register R1 gives the effective-address (B) of the operand. The data is read from location B and added to contents of register R0.

**Figure 2.11** Indirect addressing.

- To execute the Add instruction in fig 2.11 (a), the processor uses the value which is in register R1, as the EA of the operand.
- It requests a read operation from the memory to read the contents of location B. The value read is the desired operand, which the processor adds to the contents of register R0.
- Indirect addressing through a memory-location is also possible as shown in fig 2.11(b). In this case, the processor first reads the contents of memory-location A, then requests a second read operation using the

Figure 2.11 — Indirect Addressing: (a) via register, (b) via memory location

## 1.5 Index Mode

**Syntax:**  $X(R_i)$

**EA:**  $EA = X + [R_i]$

- **X** = constant value (offset / displacement) — given explicitly in the instruction.
- **R<sub>i</sub>** = index register — holds a base address or offset value.
- The contents of the index register are NOT changed during EA generation.
- **Use case 1:** R<sub>i</sub> holds base address; X is offset → access elements of an array.
- **Use case 2:** X is a memory address; R<sub>i</sub> holds offset → same result, different perspective.

**Example:**

Add 20(R1), R2 ; If R1=1000, EA = 1000 + 20 = 1020. Fetch operand at 1020, add to R2.

### 5) Index mode

- The operation is indicated as  $X(R_i)$  where X=the constant value which defines an offset(also called a displacement).  
R<sub>i</sub>=the name of the index register which contains address of a new location.
- The effective-address of the operand is given by  $EA=X+[R_i]$
- The contents of the index-register are not changed in the process of generating the effective- address.
- The constant X may be given either  
→ as an explicit number or  
→ as a symbolic-name representing a numerical value.

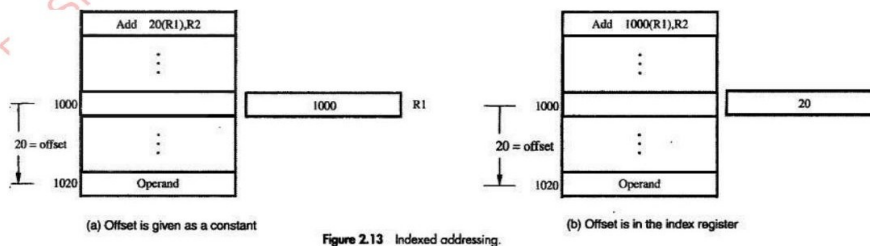


Figure 2.13 Indexed addressing.

Figure 2.13 — Indexed Addressing: (a) offset as constant, (b) offset in register

- Fig(a) illustrates two ways of using the Index mode. In fig(a), the index register, R1, contains the address of a memory-location, and the value X defines an offset(also called a displacement) from this address to the location where the operand is found.
- To find EA of operand:  
Eg: Add 20(R1), R2  
 $EA \Rightarrow 1000 + 20 = 1020$
- An alternative use is illustrated in fig(b). Here, the constant X corresponds to a memory address, and the contents of the index register define the offset to the operand. In either case, the effective-address is the sum of two values; one is given explicitly in the instruction, and the other is stored in a register.

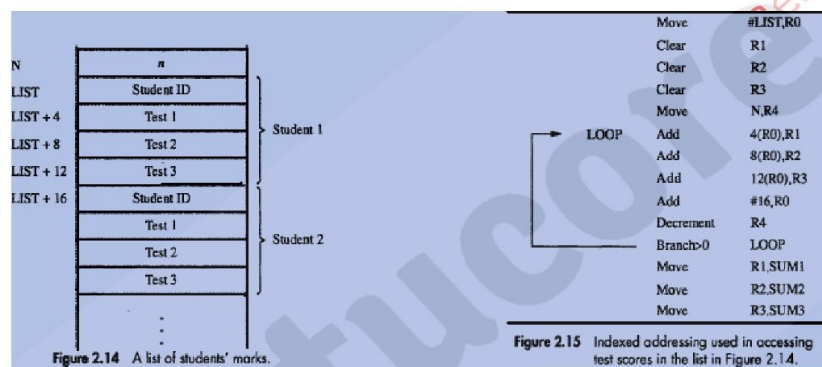


Figure 2.15 Indexed addressing used in accessing test scores in the list in Figure 2.14.

Figure 2.14 &amp; 2.15 — Student marks accessed via Index Mode

**VTU Tip:** Index mode is ideal for accessing array elements. If array starts at LIST and element size is 4, the  $i$ -th element is at  $LIST+4*i$ .

## 1.6 Base with Index Mode

**Syntax:** (R<sub>i</sub>, R<sub>j</sub>)

**EA:** EA = [R<sub>i</sub>] + [R<sub>j</sub>]

- Uses TWO registers — R<sub>i</sub> is the base register, R<sub>j</sub> holds the index/offset.
- The second register (R<sub>j</sub>) is called the BASE REGISTER.
- **Advantage:** More flexible than single-register index mode — BOTH components of EA can be changed at runtime.
- **Use:** Accessing records/structures in arrays where both the record offset and field offset can vary.

**Example:**

```
Load (R1, R2), R5 ; EA = [R1] + [R2]; fetch that memory value into R5
```

## 1.7 Base with Index & Offset Mode

**Syntax:** X(R<sub>i</sub>, R<sub>j</sub>)

**EA:** EA = X + [R<sub>i</sub>] + [R<sub>j</sub>]

- Combines an explicit constant X with TWO registers.
- **Use:** Implements 3-dimensional array access. The (R<sub>i</sub>, R<sub>j</sub>) part specifies the beginning of a record; X selects a field within that record.
- **Example use:** Accessing a field within a student record in an array of student records.

**Example:**

```
Store R5, 8(R1, R2) ; EA = 8 + [R1] + [R2]; store R5 at that memory location
```

## 1.8 Relative Mode

**Syntax:** X(PC)

**EA:** EA = [PC] + X

- Similar to Index mode, BUT uses the Program Counter (PC) instead of a general-purpose register.
- **X(PC):** Denotes an EA that is X locations above or below the CURRENT position of PC.
- Since the location is identified RELATIVE to PC, this is called Relative mode.
- **Key use:** Used in CONDITIONAL BRANCH instructions. The branch target is expressed as an offset from the current PC.
- **Advantage:** Position-independent code — programs can be loaded anywhere in memory.

**Example:**

```
Branch>0 LOOP ; If result > 0, jump to LOOP (LOOP is X bytes from current PC)
```

## 1.9 Auto Increment Mode

**Syntax:** (R<sub>i</sub>)<sup>+</sup>

**EA:** EA = [R<sub>i</sub>]; then R<sub>i</sub> ← [R<sub>i</sub>] + d (d = data size, typically 4 for 32-bit)

- EA of operand = contents of register R<sub>i</sub> (BEFORE incrementing).
- AFTER accessing the operand, R<sub>i</sub> is automatically incremented to point to the NEXT item.
- **Use:** Sequentially stepping through a list/array — R<sub>i</sub> acts as a moving pointer.
- **Advantage:** Eliminates explicit Add instruction to advance pointer.

**Example:**

```
Add (R2)+, R0    ; Add value at [R2] to R0, then increment R2 by 4 (next element)
```

## 1.10 Auto Decrement Mode

**Syntax:**  $-(R_i)$

**EA:**  $R_i \leftarrow [R_i] - d$ ; then  $EA = [R_i]$  (decrement FIRST, then access)

- $R_i$  is FIRST automatically decremented, THEN used as the EA.
- **Order:** Decrement BEFORE access (unlike Auto Increment which accesses THEN increments).
- **Use:** Together with Auto Increment, implements STACK operations — PUSH uses  $-(R_i)$ , POP uses  $(R_i)+$ .

**Example:**

```
Move NEWITEM, -(SP) ; Decrement SP first, then store NEWITEM at new SP (PUSH)
```

*VTU KEY FACT: Auto Increment + Auto Decrement together implement a STACK.  $-(SP) = \text{PUSH}$ ,  $(SP)+ = \text{POP}$ .*

## 2. Program: Add N Numbers Using Indirect Addressing Mode

Register R2 is used as a POINTER to the list. Operands are accessed indirectly through R2.

### Algorithm:

1. **Initialization:** Load counter (N) into R1, address of NUM1 into R2, clear R0.
2. **Loop:** Fetch operand at address [R2] and add to R0. Advance pointer R2 by 4. Decrement R1. Repeat while R1 > 0.
3. **Store:** Move final sum from R0 to SUM.

```
; Add N numbers using Indirect Addressing Mode
      Move  N, R1          ; Load count N into R1
      Move  #NUM1, R2      ; R2 points to first number (immediate mode
for address)
      Clear R0             ; Clear accumulator R0 = 0
LOOP:  Add   (R2), R0       ; Add value at address [R2] to R0 (INDIRECT
MODE)
      Add   #4, R2         ; Advance pointer R2 to next number (4
bytes per word)
      Decrement R1         ; Decrement counter
      Branch>0 LOOP       ; If R1 > 0, go back to LOOP
      Move  R0, SUM        ; Store result in SUM
```

**Key Point:** R2 holds the ADDRESS of the next number. The instruction `Add (R2), R0` reads the value AT that address (indirect). Each iteration, R2 is incremented by 4 (word size) to point to the next element.



### 3. Program: Add N Numbers Using Auto Increment Mode

This is more efficient than indirect mode — the pointer advance is done automatically by the hardware.

```
; Add N numbers using Indirect + Auto Increment Addressing Mode
Move N, R1      ; Load count N into R1
Move #NUM1, R2  ; R2 points to first number
Clear R0        ; Clear accumulator R0 = 0
LOOP: Add (R2)+, R0 ; Add [R2] to R0, then R2 automatically
incremented (AUTO INCREMENT)
Decrement R1    ; Decrement counter
Branch>0 LOOP   ; If R1 > 0, repeat
Move R0, SUM    ; Store result
```

- (Ri)+ ; where Ri=pointer-register.
- 10) **Auto Decrement Mode**
- The contents of a register specified in the instruction are first automatically decremented and are then used as the effective-address of the operand.
  - This mode is denoted as  $-(Ri)$  ;where Ri=pointer-register.
  - These 2 modes can be used together to implement an important data structure called a stack.

**Q. Write a program to add N numbers using indirect addressing mode.**

| Address | Contents      |                  |
|---------|---------------|------------------|
|         | Move N,R1     | } Initialization |
|         | Move #NUM1,R2 |                  |
|         | Clear R0      |                  |
| → LOOP  | Add (R2),R0   |                  |
|         | Add #4,R2     |                  |
|         | Decrement R1  |                  |
|         | Branch>0 LOOP |                  |
|         | Move R0,SUM   |                  |

Figure 2.12 Use of indirect addressing in the program of Figure 2.10.

Figure 2.12 — Indirect addressing in the add-N-numbers program

Dr. Shobha S, Cambridge Institute of Technology, Bengaluru

**Q. Write a program to add N numbers using indirect and autoincrement addressing mode.**

|       |          |                  |
|-------|----------|------------------|
| Move  | N,R1     | } Initialization |
| Move  | #NUM1,R2 |                  |
| Clear | R0       |                  |

Figure 2.16 — Auto Increment addressing in the same program

**Comparison:** In indirect mode, a separate `Add #4, R2` instruction is needed. In auto increment mode, `(R2)+` does both the access AND the increment automatically — saving one instruction per iteration.

## 4. Memory Mapped I/O vs I/O Mapped I/O

| Feature             | Memory Mapped I/O                                      | I/O Mapped I/O                                 |
|---------------------|--------------------------------------------------------|------------------------------------------------|
| Address Spaces      | ONE address space for both memory and I/O              | TWO separate address spaces for memory and I/O |
| Memory available    | Less (I/O takes some memory addresses)                 | More (I/O has its own address space)           |
| Instructions used   | Same instructions (Move/Load/Store) for memory AND I/O | Separate IN/OUT instructions for I/O           |
| Hardware complexity | Simpler (no separate I/O bus signals needed)           | Requires extra hardware for I/O select signal  |
| Efficiency          | Less efficient                                         | More efficient                                 |
| Examples            | ARM processors, most RISC systems                      | Intel x86 (uses IN/OUT instructions)           |
| Programming         | Simpler — treat I/O registers like memory              | Slightly more complex — need I/O instructions  |

*VTU Tip: In Memory Mapped I/O, a standard instruction like "MoveByte DATAIN, R1" reads a keyboard character. No special I/O instruction is needed because DATAIN is just a memory address.*

## 5. Source Program vs Object Program

| Sl. No. | Source Program                                                                  | Object Program                                                               |
|---------|---------------------------------------------------------------------------------|------------------------------------------------------------------------------|
| 1       | A collection of instructions written in a human-readable programming language   | A sequence of binary statements generated after compiling the source program |
| 2       | Contains English-like words according to the syntax of the programming language | Contains binary codes (0s and 1s) that the CPU can execute                   |
| 3       | Also called Source Code                                                         | Also called Object Code or Machine Code                                      |
| 4       | Can be read and understood by a programmer                                      | Can be read and executed by the machine/CPU                                  |
| 5       | Created by the programmer using a text editor or IDE                            | Created by the compiler or assembler automatically                           |
| 6       | The source program is the INPUT to the compiler/assembler                       | The object program is the OUTPUT of the compiler/assembler                   |

## 6. Assembler Directives

**Definition:** Assembler Directives are commands to the ASSEMBLER (not to the CPU). They are NOT translated into machine opcodes in the object program. They guide the assembler on how to assemble the program.

**Key Directives:**

### EQU (Equate)

Informs the assembler about the value of an identifier. Acts like a constant definition.

```
SUM EQU 200 ; Wherever SUM appears, replace it with 200
```

### ORIGIN

Tells the assembler the starting memory address for the next data block or code block.

```
ORIGIN 204 ; Place the following data/code starting at memory address 204
```

### DATAWORD

Tells the assembler to load (store) a specific value into the current memory location.

```
N DATAWORD 100 ; Store the value 100 at memory location labelled N (address 204)
```

### RESERVE

Reserves a block of memory for data storage without initializing it.

```
NUM1 RESERVE 400 ; Reserve 400 bytes starting at label NUM1 for number storage
```

### END

Tells the assembler that this is the END of the source program text. Nothing after END is assembled.

### RETURN

Identifies the point at which execution of the program should terminate when run.

**Label:** Any statement that causes instructions or data to be placed in memory may be given a LABEL. The label is assigned a value equal to the address of that memory location.

- EX: `NUM1 RESERVE 400` declares a memory-block of 400 bytes is to be reserved for data.
- **END** directive tells the assembler that this is the end of the source-program text.
  - **RETURN** directive identifies the point at which execution of the program should be terminated.
  - Any statement that makes instructions or data being placed in a memory-location may be given a label. The label(say N or NUM1) is assigned a value equal to the address of that location.

|                                               | Memory address label | Operation | Addressing or data information |
|-----------------------------------------------|----------------------|-----------|--------------------------------|
| Assembler directives                          | SUM                  | EQU       | 200                            |
|                                               |                      | ORIGIN    | 204                            |
|                                               | N                    | DATAWORD  | 100                            |
|                                               | NUM1                 | RESERVE   | 400                            |
| Statements that generate machine instructions |                      | ORIGIN    | 100                            |
|                                               | START                | MOVE      | N,R1                           |
|                                               |                      | MOVE      | #NUM1,R2                       |
|                                               |                      | CLR       | R0                             |
|                                               | LOOP                 | ADD       | (R2),R0                        |
|                                               |                      | ADD       | #4,R2                          |
|                                               |                      | DEC       | R1                             |
|                                               |                      | BGTZ      | LOOP                           |
| Assembler directives                          |                      | MOVE      | R0,SUM                         |
|                                               |                      | RETURN    |                                |
|                                               |                      | END       | START                          |

Figure 2.18 Assembly language representation for the program in Figure 2.17.

Figure 2.18 — Assembly language representation with directives

**VTU Exam:** Always remember — directives control the ASSEMBLER, not the CPU. They do NOT generate machine code (except DATAWORD which places data).



## 7. General Format of an Assembly Statement

Most assembly languages require statements to be written in this form:

**[ Label ] [ Operation ] [ Operands ] [ ; Comment ]**

| Field     | Optional?          | Description                                                                                                                                    |
|-----------|--------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| Label     | Optional           | A name associated with the memory address of this instruction. Used as a branch target or to reference data. Value = address of that location. |
| Operation | Required           | The mnemonic (opcode) of the machine instruction or assembler directive (e.g., MOVE, ADD, EQU, ORIGIN).                                        |
| Operands  | Required (usually) | Addressing information for one or more operands. May include registers, memory addresses, immediate values, or combinations.                   |
| Comment   | Optional           | Documentation starting with semicolon (;). Ignored by assembler. Makes program readable.                                                       |

### Example Statement:

```
LOOP:  ADD  (R2)+, R0    ; Add value at [R2] to R0, auto-increment R2
```

**LOOP** = Label (marks this instruction's address), **ADD** = Operation, **(R2)+, R0** = Operands, **; Add value...** = Comment

## 8. Assembler & Its Functions

**Assembler:** A program that automatically translates assembly language programs into machine language (binary) instructions.

### Functions of the Assembler:

4. Replaces all symbols denoting operations and addressing modes with their binary codes used in machine instructions.
5. Replaces all names and labels with their actual numerical memory addresses.
6. Assigns addresses to instructions and data blocks, starting at the address given in the ORIGIN directive.
7. Inserts constants and initial values that are specified in DATAWORD directives.
8. Reserves memory space as requested by RESERVE directives.
9. Generates the object program (binary) and usually stores it on disk for later execution.

*Summary: Assembler = Source Program → Object Program. It does symbol table management, address assignment, and binary code generation.*

## 9. Two Pass Assembler

**Problem with Single Pass:** When an instruction references a FORWARD label (a label defined LATER in the program), the assembler cannot resolve the address in a single scan.

### Solution — Two Pass Assembler:

#### Pass 1: Build Symbol Table

- Assembler scans through the entire source program from beginning to end.
- It keeps track of ALL names (labels) and assigns each a numerical value = its memory address.
- All these name-address pairs are stored in a SYMBOL TABLE.
- No machine code is generated in Pass 1 — only the symbol table is built.

#### Pass 2: Generate Machine Code

- Assembler scans the source program AGAIN.
- Whenever a name/label appears, it is REPLACED with its value from the symbol table.
- Machine binary code is generated for each instruction.
- The final object program is produced and stored on disk.

*Key Point: Two passes are needed because a branch instruction may reference a label that appears LATER (forward reference). Pass 1 collects all addresses; Pass 2 uses them to generate correct binary.*

## 10. Loader & Debugger Programs

### Loader Program

- The assembler stores the object program (binary) on magnetic disk.
- **Loader Program** is used to LOAD the object program from disk into the MEMORY of the computer before it is executed.
- It handles memory allocation and address relocation if needed.

### Debugger Program

- **Debugger Program** is used to help the user find PROGRAMMING ERRORS (bugs) in their program.
- It enables the user to:

- → STOP execution of the object program at specific points of interest (called breakpoints)
- → EXAMINE the contents of various processor registers and memory locations at any point
- → STEP through the program one instruction at a time to trace execution
- Essential tool in the software development process — without it, finding bugs in binary programs would be nearly impossible.



## 11. Basic Input/Output Operations

Consider transferring a character code from the keyboard to the processor. Two hardware components are involved:

| Register/Flag     | Role                                                                                                              |
|-------------------|-------------------------------------------------------------------------------------------------------------------|
| DATAIN            | Buffer register for KEYBOARD input — stores the ASCII code of the last key pressed                                |
| SIN (Status IN)   | Flag bit for keyboard: SIN=1 means a valid character is waiting in DATAIN; SIN=0 means character was already read |
| DATAOUT           | Buffer register for DISPLAY output — character placed here is sent to screen                                      |
| SOUT (Status OUT) | Flag bit for display: SOUT=1 means display is READY to receive a new character; SOUT=0 means it is busy           |

### Keyboard → Processor:

10. User presses a key → ASCII code stored in DATAIN → SIN set to 1.
11. Processor POLLS SIN continuously (checks if SIN=1).
12. When SIN=1, processor reads DATAIN → SIN automatically cleared to 0.

### Processor → Display:

13. Processor POLLS SOUT (checks if SOUT=1 = display is ready).
14. When SOUT=1, processor writes character to DATAOUT → SOUT automatically cleared to 0.
15. Display reads DATAOUT and shows the character.

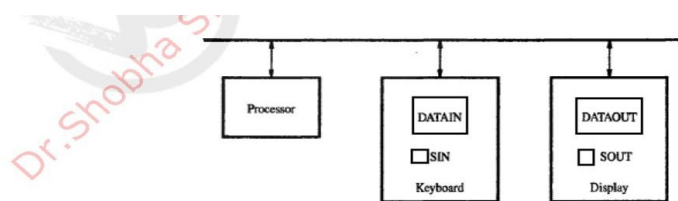


Figure 2.19 Bus connection for processor, keyboard, and display.

7

Figure 2.19 — Bus connection for Processor, Keyboard, and Display

**Device Interface:** The registers DATAIN, DATAOUT and flags SIN, SOUT are part of circuitry known as a **device interface** — the hardware that connects the CPU bus to the peripheral device.

*VTU Tip: This is PROGRAMMED I/O (Polling/Busy-wait). The CPU keeps checking the status flag in a loop — simple but wastes CPU time.*

## 12. Program: Read a Line of Characters & Display It

Key instructions used:

- **MoveByte:** Transfers a BYTE (8-bit character) instead of a full word.
- **TestBit:** Tests a specific bit in a register/flag. Used to check SIN/SOUT status.

|      |          |              |                                                                                                                               |
|------|----------|--------------|-------------------------------------------------------------------------------------------------------------------------------|
|      | Move     | #LOC,R0      | Initialize pointer register R0 to point to the address of the first location in memory where the characters are to be stored. |
| READ | TestBit  | #3,INSTATUS  | Wait for a character to be entered in the keyboard buffer DATAIN.                                                             |
|      | Branch=0 | READ         |                                                                                                                               |
|      | MoveByte | DATAIN,(R0)  | Transfer the character from DATAIN into the memory (this clears SIN to 0).                                                    |
| ECHO | TestBit  | #3,OUTSTATUS | Wait for the display to become ready.                                                                                         |
|      | Branch=0 | ECHO         |                                                                                                                               |
|      | MoveByte | (R0),DATAOUT | Move the character just read to the display buffer register (this clears SOUT to 0).                                          |
|      | Compare  | #CR,(R0)+    | Check if the character just read is CR (carriage return). If it is not CR, then branch back and read another character.       |
|      | Branch≠0 | READ         | Also, increment the pointer to store the next character.                                                                      |

Figure 2.20 A program that reads a line of characters and displays it.

**Q. What is Stack? Explain stack operations with neat diagram. OR**

**Q. Define Stack. Explain PUSH and POP operations on stack with neat sketches and examples.**

- A **stack** is a special type of data structure where elements are inserted from one end and elements are deleted from the same end. This end is called the **top** of the stack (Figure: 2.14).
- The various operations performed on stack:

Figure 2.20 — Program that reads a line of characters and displays it

**Program explanation step by step:**

16. **Move #LOC, R0:** Initialize R0 to point to the memory buffer where characters will be stored.
17. **READ section:** TestBit #3, INSTATUS — checks SIN flag. If SIN=0 (no character ready), loop back to READ (busy-wait). When SIN=1, MoveByte DATAIN,(R0) — read character from keyboard into memory at [R0].
18. **ECHO section:** TestBit #3, OUTSTATUS — checks SOUT flag. If SOUT=0 (display busy), loop back to ECHO. When SOUT=1, MoveByte (R0),DATAOUT — send the character to display.
19. **Compare #CR,(R0)+:** Check if the character just read is a Carriage Return (CR). If NOT CR, branch back to READ to read the next character. R0 is also auto-incremented to store the next character in the next memory location.
20. **If CR detected:** The loop ends — the entire line has been read and echoed.

## 13. Stack – Definition & Operations (PUSH & POP)

**Definition:** A Stack is a special type of data structure where elements are inserted and deleted from the SAME end called the **TOP of the stack**. It follows the **LIFO (Last In, First Out)** principle.

**Stack Pointer (SP):** A processor register that keeps track of the address of the current TOP element of the stack at all times.

Dr. Shobha S, Cambridge Institute of Technology, Bengaluru

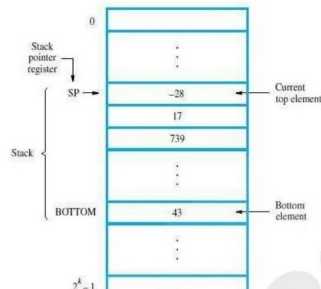


Figure 2.14 A stack of words in the memory.

**Q. Write a program that checks for empty and full errors in pop and push operations. OR**  
**Q. Write a program routine for a safe push and safe pop operations.**

SAFEPOP    Compare    #2000,SP    Check to see if the stack pointer contains  
                   Branch    UNDEFVDDDD    an address value smaller than 2000. If it

Figure 2.14 — A stack of words in memory

### PUSH Operation (Insert)

Adding a new element onto the TOP of the stack. The SP is decremented FIRST (stack grows downward in memory), then the item is written.

```
; PUSH operation (byte-addressable, 32-bit word)
Subtract #4, SP      ; Decrement SP by 4 (move top pointer up/down)
Move NEWITEM, (SP)   ; Write NEWITEM to the new top of stack
```

**Equivalent:** Move NEWITEM, -(SP) — Using Auto Decrement mode (combined in one instruction)

### POP Operation (Delete)

Removing the TOP element from the stack. The item is read first, then SP is incremented.

```
; POP operation
Move (SP), ITEM      ; Read the top-of-stack value into ITEM
Add #4, SP           ; Increment SP by 4 (shrink the stack)
```

**Equivalent:** Move (SP)+, ITEM — Using Auto Increment mode (combined in one instruction)

**IMPORTANT:** Stack grows **DOWNWARD** in memory (towards lower addresses). **PUSH** decrements SP, **POP** increments SP. Top of stack is always pointed to by SP.

## 14. Safe PUSH & POP Operations (With Error Checking)

Normal PUSH/POP operations do not check for stack overflow or underflow. Safe versions add boundary checks:

| ROUTINE                               | STATE & COMMENTS   | ON ADDRESS VALUE BEYOND BOUND. AS IT DOES, THE STACK IS EMPTY. BRANCH TO THE ROUTINE EMPTYERROR FOR APPROPRIATE ACTION.                                                        |
|---------------------------------------|--------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Move                                  | (SP)+, ITEM        | Otherwise, pop the top of the stack into memory location ITEM.                                                                                                                 |
| (a) Routine for a safe pop operation  |                    |                                                                                                                                                                                |
| SAFEPUSH                              | Compare #1500, SP  | Check to see if the stack pointer contains an address value equal to or less than 1500. If it does, the stack is full. Branch to the routine FULLERROR for appropriate action. |
|                                       | Branch≤0 FULLERROR |                                                                                                                                                                                |
| Move                                  | NEWITEM, -(SP)     | Otherwise, push the element in memory location NEWITEM onto the stack.                                                                                                         |
| (b) Routine for a safe push operation |                    |                                                                                                                                                                                |

Figure 2.23 Checking for empty and full errors in pop and push operations.

9

Figure 2.23 — Safe PUSH (SAFEPUSH) and Safe POP (SAFEPOP) routines

### SAFEPOP (Safe POP):

```

SAFEPOP: Compare #2000, SP      ; Check if SP > 2000 (stack is EMPTY)
          Branch>0 EMPTYERROR  ; If yes, stack is empty → handle
error
          Move      (SP)+, ITEM  ; Otherwise, pop item from stack
          (auto increment)

```

**Logic:** If  $SP > 2000$  (the top boundary), the stack is EMPTY — nothing to pop. The EMPTYERROR routine handles this case.

### SAFEPUSH (Safe PUSH):

```

SAFEPUSH: Compare #1500, SP      ; Check if SP ≤ 1500 (stack is FULL)
          Branch≤0 FULLERROR    ; If yes, stack is full → handle
error
          Move      NEWITEM, -(SP) ; Otherwise, push item onto stack
          (auto decrement)

```

**Logic:** If  $SP \leq 1500$  (the bottom boundary), the stack is FULL — no more space to push. The FULLERROR routine handles this case.

## 15. Reverse Polish Notation (RPN)

**Definition:** Reverse Polish Notation (RPN), also called **Postfix notation**, is a method for writing mathematical expressions where the **operator follows its operands**. This eliminates the need for brackets and parentheses to define evaluation priority.

| Notation | Name                          | Form                      | Example (A+B) |
|----------|-------------------------------|---------------------------|---------------|
| Infix    | Standard/Infix                | Operator between operands | A + B         |
| Prefix   | Polish Notation               | Operator before operands  | + A B         |
| Postfix  | Reverse Polish Notation (RPN) | Operator after operands   | A B +         |

### Why RPN is useful for computers:

- Unambiguous — no need for parentheses or operator precedence rules.
- Easy to evaluate using a stack — operands are pushed; when an operator is found, pop operands and push result.
- Compilers convert infix to RPN internally for expression evaluation.

### RPN Evaluation Example:

Expression:  $A * B + C * D$

RPN:  $A \ B \ * \ C \ D \ * \ +$

Stack-based evaluation:

Push A, Push B → Pop B, A → Push ( $A * B$ )

Push C, Push D → Pop D, C → Push ( $C * D$ )

Pop ( $C * D$ ), ( $A * B$ ) → Push ( $A * B + C * D$ )

Numerical example:  $2 * 4 + 3 * 3$

$= (2)(4) * (3)(3) * +$

$= (8)(9) +$

$= 17$

Dr. Shobha S. Cambridge Institute of Technology, Bengaluru

**Q. Write a note on Reverse Polish Notation.****Reverse Polish Notation**

- Infix Notation  
 $A + B$
- Prefix or Polish Notation  
 $+ A B$
- Postfix or Reverse Polish Notation (RPN)  
 $A B +$

 $A * B + C * D \xrightarrow{\text{RPN}} A B * C D * +$ 

|     |     |     |     |     |   |   |
|-----|-----|-----|-----|-----|---|---|
| (2) | (4) | *   | (3) | (3) | * | + |
| (8) | (3) | (3) | *   | +   |   |   |
| (8) | (9) | +   |     |     |   |   |
| 17  |     |     |     |     |   |   |

**Reverse Polish Notation**

- Example

$(A + B) * [C * (D + E) + F]$   
 $(A B +) (D E +) C * F + *$

Reverse Polish notation (RPN) is a method for conveying mathematical expressions without the use of separators such as brackets and parentheses. In this notation, the operators follow

Figure — Reverse Polish Notation examples and expression tree

## 16. Queue – Definition & Operations

**Definition:** A Queue is an ordered group of homogeneous elements where data is stored and retrieved on a **FIFO (First In, First Out)** basis. Elements are added at the REAR and removed from the FRONT.

**Operations:**

- **Enqueue (Insert):** Add a new element at the REAR end of the queue.
- **Dequeue (Delete):** Remove an element from the FRONT end of the queue.

**Two Pointers Required:**

- **FRONT pointer:** Points to the element to be dequeued next.
- **REAR pointer:** Points to the last element enqueued; new elements go after this.

**Problem:** Without control, a queue moves continuously toward higher memory addresses. Solution: use a **Circular Buffer** — when REAR reaches the end of the allocated memory, it wraps around to the beginning.

**Circular Buffer:** REAR pointer wraps around to address 0 after reaching the maximum address of the queue region. This keeps the queue in a fixed memory region.

## 17. Stack vs Queue

| Feature          | Stack                                       | Queue                                              |
|------------------|---------------------------------------------|----------------------------------------------------|
| Principle        | LIFO (Last In, First Out)                   | FIFO (First In, First Out)                         |
| Insert operation | PUSH (from TOP)                             | Enqueue (at REAR)                                  |
| Delete operation | POP (from TOP)                              | Dequeue (from FRONT)                               |
| Access           | Only TOP is accessible                      | FRONT for deletion, REAR for insertion             |
| Pointers needed  | 1 pointer (Stack Pointer SP)                | 2 pointers (FRONT and REAR)                        |
| Implementation   | Simple                                      | More complex                                       |
| Empty condition  | Top == -1                                   | Front == -1 or Front == Rear+1                     |
| Full condition   | Top == Max-1                                | Rear == Max-1                                      |
| Variants         | None                                        | Circular queue, Double-ended queue, Priority queue |
| Visual model     | Vertical (like a pile of plates)            | Horizontal (like a line of people)                 |
| Use cases        | Recursion, expression evaluation, undo/redo | Scheduling, BFS, I/O buffering                     |



## 18. Subroutines – Call & Return

**Definition:** A subtask consisting of a set of instructions which is executed many times (possibly from different parts of the main program) is called a **Subroutine** (also called a procedure or function in high-level languages).

### Key Benefits:

- Code reuse — write once, call many times.
- Modular programming — break complex tasks into manageable pieces.
- Reduces code size and improves maintainability.

**Subroutine Linkage:** The mechanism by which the computer calls and returns from subroutines. The simplest method uses a **Link Register**.

### CALL Instruction:

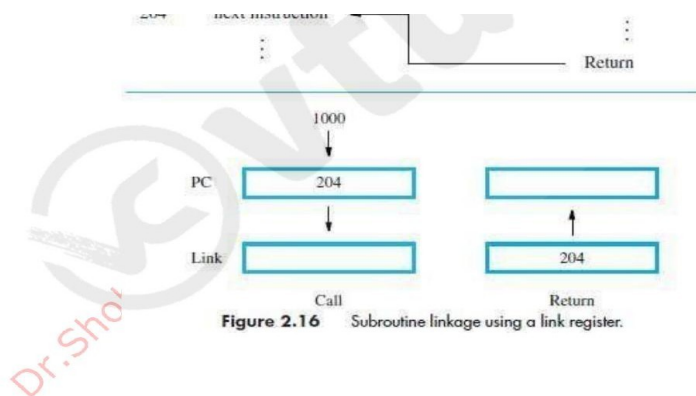
A special branch instruction that performs two operations atomically:

21. **Store PC** (return address = address of next instruction after Call) into the **Link Register**
22. **Branch** to the TARGET address of the subroutine

### RETURN Instruction:

A special branch instruction that:

23. **Branches to the address CONTAINED in the Link Register** (which was saved by the most recent CALL)



12

Figure 2.16 — Subroutine linkage using a Link Register

### Execution flow example:

```
Memory 200: Call SUB          ; Store PC(=204) in Link, jump to address
1000
Memory 204: next instruction ; ← Program resumes here after Return
...
Memory 1000: first instruction ; ← Subroutine SUB starts here
...
          Return              ; Branch to address in Link Register (=
204)
```

## 19. Subroutine Nesting & Processor Stack

**Subroutine Nesting:** When one subroutine calls another subroutine, this is called **subroutine nesting**.

**Problem with Link Register:** When a second subroutine is called, the return address for the second call **OVERWRITES** the Link Register, destroying the return address of the first subroutine. This makes correct return impossible.

**Solution — Processor Stack:**

- Return addresses are pushed onto a STACK (using SP register).
- **CALL instruction:** Pushes the contents of PC (return address) onto the processor stack.
- **RETURN instruction:** Pops the return address from the top of the processor stack into PC.
- Since return addresses are used in LIFO order (last call returns first), a stack is the PERFECT data structure.
- Nesting can go to ANY depth — each call pushes a new return address; each return pops the most recent one.

```
; Nested subroutine example
Main:   Call SUB1       ; PC(return addr) pushed onto stack
...
SUB1:   Call SUB2       ; Return addr of SUB1 pushed onto stack
...
SUB2:   Return          ; Pop return addr → go back to SUB1
...
SUB1:   Return          ; Pop return addr → go back to Main
```

*VTU KEY: Subroutine nesting requires a STACK. A simple Link Register cannot support nesting because the second Call overwrites the first return address. Stack-based CALL/RETURN supports nesting AND recursion.*

## 20. Parameter Passing

**Definition:** The exchange of information between a calling program and a subroutine is referred to as **Parameter Passing**. The caller passes input values (parameters) and receives output values (results) through agreed-upon locations.

### Methods of Parameter Passing:

24. **Via Registers (most common):** Parameters are placed in specific CPU registers before calling the subroutine. The subroutine reads from and writes to these registers.
25. **Via Memory locations:** Parameters are stored at specific memory locations that both caller and subroutine know about.
26. **Via Processor Stack:** Parameters are pushed onto the stack before the CALL. The subroutine pops them off the stack.

by the subroutine.

- Alternatively, parameters may be placed on the processor-stack used for saving the return-address.
- Following is a program for adding a list of numbers using subroutine with the parameters passed through registers.

| Calling program |           |          |                            |
|-----------------|-----------|----------|----------------------------|
| Move            | N,R1      |          | R1 serves as a counter.    |
| Move            | #NUM1,R2  |          | R2 points to the list.     |
| Call            | LISTADD   |          | Call subroutine.           |
| Move            | R0,SUM    |          | Save result.               |
| :               |           |          |                            |
| Subroutine      |           |          |                            |
| LISTADD         | Clear     | R0       | Initialize sum to 0.       |
| LOOP            | Add       | (R2)+,R0 | Add entry from list.       |
|                 | Decrement | R1       |                            |
|                 | Branch>0  | LOOP     |                            |
|                 | Return    |          | Return to calling program. |

**Figure 2.25** Program of Figure 2.16 written as a subroutine; parameters passed through registers.

13

Figure 2.25 — Parameter passing via registers (Add N numbers as subroutine)

### Example — Calling program:

```
; Calling Program
Move  N, R1          ; R1 = count (parameter)
Move  #NUM1, R2      ; R2 = address of list (parameter)
Call  LISTADD        ; Call the subroutine
Move  R0, SUM        ; R0 = result (returned by subroutine)
```

### Example — Subroutine:

```
; Subroutine LISTADD
LISTADD: Clear  R0          ; Initialize sum = 0
LOOP:   Add     (R2)+, R0    ; Add next element, advance pointer
        Decrement R1        ; Decrement counter
        Branch>0 LOOP      ; Repeat if more elements
        Return              ; Return to calling program (R0 has the
sum)
```

## 21. Logical Shift Instructions

**Purpose:** Shift the bits of an operand left or right by a specified number of positions. Used for multiplication/division by powers of 2, bit manipulation, and packing/unpacking data.

**Two types of shifts:**

- **Logical Shift:** Used for GENERAL (unsigned) binary-coded data. Vacated positions are filled with 0.
- **Arithmetic Shift:** Used for SIGNED NUMBERS. Preserves the sign bit — fills vacated positions with the sign bit (for right shift).

### Logical Shift Left (LShiftL):

- Bits shifted LEFT; zeros fill from RIGHT; leftmost bit goes into Carry flag C.
- Effect: Multiplies the value by 2 for each position shifted (for unsigned numbers).

```
LShiftL R3, #2 ; Shift R3 left by 2 positions (multiply by 4)
```

### Logical Shift Right (LShiftR):

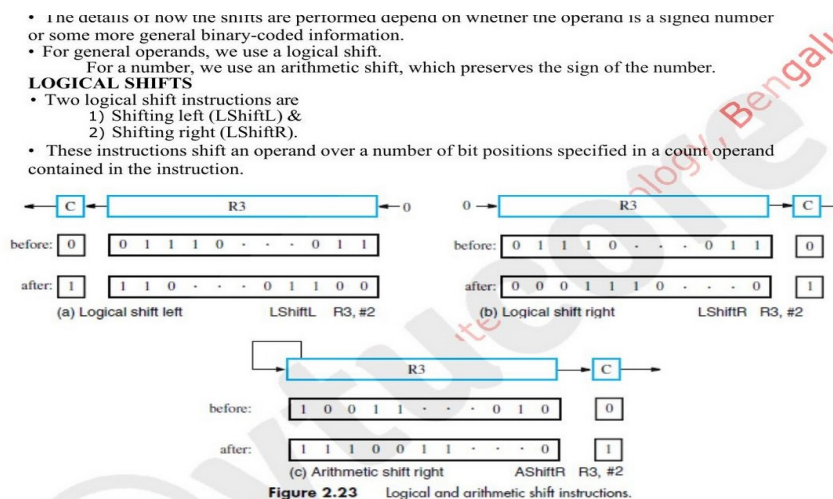
- Bits shifted RIGHT; zeros fill from LEFT; rightmost bit goes into Carry flag C.
- Effect: Divides the value by 2 for each position shifted (for unsigned numbers).

```
LShiftR R3, #2 ; Shift R3 right by 2 positions (divide by 4)
```

### Arithmetic Shift Right (AShiftR):

- Bits shifted RIGHT; SIGN BIT replicates to fill from LEFT (preserves sign for negative numbers).
- Rightmost bit goes into Carry flag C.

```
AShiftR R3, #2 ; Arithmetic shift right by 2 (signed divide by 4)
```



**Q. Write a program that packs two BCD digits using logical shift instruction.**

```
Move    #LOC,R0    R0 points to data.
MoveByte (R0)+,R1    Load first byte into R1.
LShiftL #4,R1      Shift left by 4 bit positions.
```

Figure 2.23 — Logical shift left, logical shift right, and arithmetic shift right

**Key difference:** LShiftR fills with 0 from left. AShiftR fills with sign bit (0 for positive, 1 for negative). For positive numbers both give same result.

## 22. Program: Pack Two BCD Digits

**Problem:** Two BCD digits are stored as separate bytes (each 4 bits used). Pack them into a single byte — high nibble gets first digit, low nibble gets second.

**Steps:**

27. Load first BCD digit byte into R1.
28. Shift R1 LEFT by 4 positions — moves the digit to the HIGH nibble position.
29. Load second BCD digit byte into R2.
30. AND R2 with 0x0F (mask off the high nibble of R2, keep only low 4 bits).
31. OR R1 with R2 — combine the two nibbles into one byte.
32. Store the result.

Dr. Shobh

|          |            |                             |
|----------|------------|-----------------------------|
| MoveByte | (R0), R2   | Load second byte into R2.   |
| And      | #\$F, R2   | Eliminate high-order bits.  |
| Or       | R1, R2     | Concatenate the BCD digits. |
| MoveByte | R2, PACKED | Store the result.           |

**Figure 2.31** A routine that packs two BCD digits.

**Q. Explain the types of rotate instructions with example for each.**

- In shift operations, the bits shifted out of the operand are lost, except for the last bit shifted out which is retained in the Carry-flag C.
- To preserve all bits, a set of rotate instructions can be used.
- They move the bits that are shifted out of one end of the operand back into the other end.
- Two versions of both the left and right rotate instructions are usually

14

*Figure 2.31 — Routine that packs two BCD digits*

```
; Pack two BCD digits into one byte
Move      #LOC, R0      ; R0 points to data location
MoveByte  (R0)+, R1     ; Load first BCD digit byte into R1, advance
pointer
LShiftL   #4, R1        ; Shift R1 left 4 positions (first digit →
high nibble)
MoveByte  (R0), R2      ; Load second BCD digit byte into R2
And       #$F, R2       ; Mask: keep only low 4 bits of R2 (eliminate
upper bits)
Or        R1, R2        ; Combine: R2 = first_digit(high) |
second_digit(low)
MoveByte  R2, PACKED    ; Store the packed BCD byte
```

## 23. Rotate Instructions

**Problem with Shift:** In shift operations, bits shifted out of the operand are LOST (except the last bit which goes to Carry flag C). To PRESERVE all bits, rotate instructions are used.

**How Rotate works:** Bits shifted out of one end of the operand are fed BACK INTO the other end. No bits are lost.

**Four rotate variants:**

| Instruction                  | Syntax          | Description                                                                                  |
|------------------------------|-----------------|----------------------------------------------------------------------------------------------|
| Rotate Left (without carry)  | RotateL Ri, #n  | Bits rotate left n positions. MSB goes to LSB position. C gets a copy of the bit rotated.    |
| Rotate Left with Carry       | RotateLC Ri, #n | C bit is included in the rotation. The bit that exits the MSB goes into C; old C enters LSB. |
| Rotate Right (without carry) | RotateR Ri, #n  | Bits rotate right n positions. LSB goes to MSB position. C gets a copy of the bit rotated.   |
| Rotate Right with Carry      | RotateRC Ri, #n | C bit included. Bit exits LSB into C; old C enters MSB.                                      |

Dr. Shobha S., Cambridge Institute of Technology, Bengaluru

- provided. In one version, the bits of the operand is simply rotated. In the other version, the rotation includes the C flag.

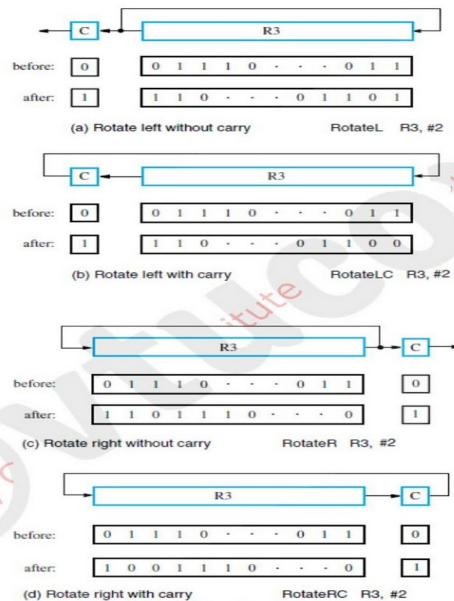


Figure 2.25 Rotate instructions.

15

Figure 2.25 — All four rotate instructions (left/right, with/without carry)

**Use cases:**

- Rotate is useful when processing data in blocks — e.g., serial communication, CRC computation.
- Rotate-with-carry allows multi-word shifts by chaining through the Carry flag.
- Example:** To pack multiple bytes: rotate each byte into position without losing bits.



## 24. Solved Problems

*These 8 problems cover all major VTU exam patterns. Study EA calculations and program writing carefully.*

### Problem 1: Evaluate $A*B + C*D$ (Single Accumulator Processor)

**Question:** Write a program that evaluates  $A*B+C*D$  on a processor with a single accumulator. Available instructions: Load, Store, Multiply, Add.

**Solution:**

```
; Evaluate A*B + C*D using single accumulator (accumulator = implied register)
Load      A           ; Accumulator ← [A]
Multiply  B           ; Accumulator ← Accumulator * [B] (= A*B)
Store     RESULT      ; RESULT ← A*B (save partial result)
Load      C           ; Accumulator ← [C]
Multiply  D           ; Accumulator ← Accumulator * [D] (= C*D)
Add       RESULT      ; Accumulator ← C*D + RESULT = C*D + A*B
Store     RESULT      ; RESULT ← A*B + C*D (final answer)
```

**Explanation:** A single-accumulator processor has only one register for arithmetic. We must compute  $A*B$  first, save it, then compute  $C*D$ , and finally add the saved  $A*B$ .

### Problem 2: Effective Address Calculation ( $R1=1200$ , $R2=4600$ )

**Question:** Find the EA for each instruction given  $R1=1200$ ,  $R2=4600$ :

```
(a) Load 20(R1), R5    → Index mode:      EA = [R1] + 20 = 1200 + 20 = 1220
(b) Move #3000, R5      → Immediate mode:   EA = 3000 (operand IS 3000, not an address)
(c) Store R5, 30(R1,R2) → Base+Index+Offset: EA = [R1]+[R2]+30 = 1200+4600+30 = 5830
(d) Add -(R2), R5       → Auto Decrement:    R2 decremented by 1
first → EA = 4600-1 = 4599
(e) Subtract (R1)+, R5  → Auto Increment:    EA = [R1] = 1200, then R1 incremented
```

### Problem 3: Effective Address Calculation ( $R1=2900$ , $R2=3300$ )

```
(a) Load R1, 55(R2)    → Index mode:      EA = 55 + [R2] = 55 + 3300 = 3355
(b) Move #2000, R7      → Immediate mode:   EA = 2000 (the value 2000 is placed in R7)
(c) Store 95(R1,R2), R5 → Base+Index+Offset: EA = 95+[R1]+[R2] = 95+2900+3300 = 6295
(d) Add (R1)+, R5       → Auto Increment:    EA = [R1] = 2900, then R1 incremented by 4
(e) Subtract -(R2), R5  → Auto Decrement:    R2 decremented by 4
first → EA = 3300-4 = 3296
```

### Problem 4: Binary Pattern — Instruction or Number?

**Question:** Given a binary pattern in a memory location, is it possible to tell whether it represents a machine instruction or a number?

**Solution: NO.** Any binary pattern can be interpreted as EITHER a number OR a machine instruction. The CPU uses it as an instruction only when the PC points to it during fetch; otherwise it



is treated as data. The bits themselves have no intrinsic meaning — context (how they are used) determines their interpretation.

### Problem 5: ORIGIN/DATAWORD vs Move Instruction

**Question:** Both ORIGIN 1000 / DATAWORD 300 and Move #300, 1000 store 300 at location 1000. Explain the difference.

**Solution:**

- **ORIGIN 1000 + DATAWORD 300:** These are ASSEMBLER DIRECTIVES. They cause the assembler to put 300 into the object program's memory image at address 1000. The value 300 is placed at location 1000 when the program is LOADED into memory (before execution begins). It happens at LOAD TIME.
- **Move #300, 1000:** This is a MACHINE INSTRUCTION. It places 300 into memory location 1000 when this instruction is EXECUTED during program run. It happens at RUN TIME (EXECUTION TIME).

### Problem 6: Stack Operations Using Addressing Modes

**Question:** R5 points to the top of a stack. Write instructions to: Pop the top two items, add them, push the result.

```
; Pop top two items, add them, push result
Move (R5)+, R0    ; Pop first item into R0 (Auto Increment: read
[R5], then R5+=4)
Add (R5)+, R0     ; Pop second item and add to R0 (Auto Increment
again)
Move R0, -(R5)    ; Push result back onto stack (Auto Decrement: R5-
=4, write R0)
```

**Explanation:** (R5)+ pops = read from current SP then increment. -(R5) pushes = decrement then write. After 2 pops and 1 push, the net change to SP is +4 (stack is 1 item shorter, which is correct — 2 items replaced by 1).

### Problem 7: Subroutine Return Address — Which Supports Nesting/Recursion?

**Question:** Three options for saving return address: (a) processor register, (b) memory location at the call site, (c) stack. Which supports nesting and recursion?

| Option                                 | Nesting Supported? | Recursion Supported? | Reason                                                                                                                                                                     |
|----------------------------------------|--------------------|----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| (a) Processor register (Link Register) | NO                 | NO                   | A second Call overwrites the return address — first return address is lost                                                                                                 |
| (b) Memory location at call site       | YES                | NO                   | Different Call instructions use different memory locations, so nesting works. But recursion (same subroutine calls itself) reuses same Call location → address overwritten |
| (c) Stack                              | YES                | YES                  | Each Call pushes a new return address; each Return pops the last one. LIFO order perfectly matches call/return order, supporting unlimited nesting and recursion           |

### Problem 8: ALP using Auto Increment Mode — Sum of N Numbers

**Question:** Numbers stored at TABLE, count at LOCN. Compute sum using auto increment mode, store at ResultL.

```
; ALP: Sum of N numbers stored at TABLE using Auto Increment mode
```

```
Move    LOCN, R1      ; R1 = count N (loop counter)
Move    #TABLE, R2    ; R2 = pointer to start of TABLE
Clear   R3             ; R3 = accumulator (sum = 0)
Label:  Add    (R2)+, R3 ; R3 += [R2], then R2 auto-incremented to
next element
        Decrement R1   ; R1--
        Branch>0 Label ; If R1 > 0, repeat loop
        Store  R3, RESULTL ; Store final sum at RESULTL
```

**Note:** (R2)+ is Auto Increment mode — reads the value at [R2] and simultaneously increments R2 by 4 (word size). This is more efficient than using a separate Add #4, R2 instruction.

## 25. Quick Revision Summary

*All key facts, formulas and tables for last-minute exam prep — vtu-easy.com*

### Addressing Modes — Quick Reference

| Mode              | Syntax        | EA Formula               | Key Use                                       |
|-------------------|---------------|--------------------------|-----------------------------------------------|
| Register          | Ri            | $EA = Ri$                | Fast temp storage; no memory access           |
| Absolute (Direct) | LOC           | $EA = LOC$               | Global variables; full address in instruction |
| Immediate         | #Value        | Operand = Value          | Constants; source only, no EA                 |
| Indirect          | (Ri) or (LOC) | $EA = [Ri]$ or $[[LOC]]$ | Pointers; variable address at runtime         |
| Index             | X(Ri)         | $EA = X + [Ri]$          | Array access; X=offset, Ri=base               |
| Base+Index        | (Ri,Rj)       | $EA = [Ri] + [Rj]$       | Records in arrays; both components variable   |
| Base+Index+Offset | X(Ri,Rj)      | $EA = X + [Ri] + [Rj]$   | 3D arrays; field in record in array           |
| Relative          | X(PC)         | $EA = [PC] + X$          | Branch targets; position-independent code     |
| Auto Increment    | (Ri)+         | $EA = [Ri]; Ri += d$     | Sequential list traversal (forward)           |
| Auto Decrement    | -(Ri)         | $Ri -= d; EA = [Ri]$     | Stack PUSH; sequential traversal (backward)   |

### Assembly Language — Key Directives

| Directive | Purpose                     | Example          |
|-----------|-----------------------------|------------------|
| EQU       | Define a constant name      | SUM EQU 200      |
| ORIGIN    | Set starting memory address | ORIGIN 204       |
| DATAWORD  | Store a value in memory     | N DATAWORD 100   |
| RESERVE   | Reserve memory block        | NUM1 RESERVE 400 |
| END       | End of source program       | END              |
| RETURN    | End program execution       | RETURN           |

### Stack & Subroutine — Key Facts

| Concept         | Key Fact                                         |
|-----------------|--------------------------------------------------|
| Stack principle | LIFO — Last In, First Out                        |
| PUSH operation  | Sub #4,SP then Move ITEM,(SP) OR Move ITEM,-(SP) |
| POP operation   | Move (SP),ITEM then Add #4,SP OR Move (SP)+,ITEM |
| Stack grows     | DOWNWARD (toward lower addresses)                |

|                     |                                                             |
|---------------------|-------------------------------------------------------------|
| SP after PUSH       | SP decreases (points to new top)                            |
| SP after POP        | SP increases (points to item below old top)                 |
| Subroutine CALL     | Saves PC to Link Register (or stack); jumps to subroutine   |
| Subroutine RETURN   | Branches to address in Link Register (or pops from stack)   |
| Nesting requirement | STACK (not link register) — each call pushes return address |
| Parameter passing   | Via registers (fastest), memory locations, or stack         |
| Safe PUSH check     | If $SP \leq \text{BOTTOM}$ → FULLERROR (stack is full)      |
| Safe POP check      | If $SP > \text{TOP}$ → EMPTYERROR (stack is empty)          |

## Shift & Rotate — Quick Reference

| Instruction | Direction | Fill bits               | Carry         | Use                          |
|-------------|-----------|-------------------------|---------------|------------------------------|
| LShiftL     | Left      | 0 from right            | Gets MSB      | Multiply by $2^n$ (unsigned) |
| LShiftR     | Right     | 0 from left             | Gets LSB      | Divide by $2^n$ (unsigned)   |
| AShiftR     | Right     | Sign bit from left      | Gets LSB      | Divide by $2^n$ (signed)     |
| RotateL     | Left      | MSB wraps to LSB        | Gets MSB copy | Bit rotation, no data loss   |
| RotateLC    | Left      | C wraps to LSB, MSB → C | Cycled        | Multi-word left shift        |
| RotateR     | Right     | LSB wraps to MSB        | Gets LSB copy | Bit rotation, no data loss   |
| RotateRC    | Right     | C wraps to MSB, LSB → C | Cycled        | Multi-word right shift       |

## Frequently Asked VTU Questions

| Question (Marks)                            | Key Points to Cover                                           |
|---------------------------------------------|---------------------------------------------------------------|
| Define & explain 4 addressing modes (8–10M) | Table 2.1 + each mode: syntax, EA formula, example, use case  |
| All 10 addressing modes (12–15M)            | All modes with diagrams + Table 2.1                           |
| Add N numbers — indirect mode (6M)          | Code + explanation of R2 as pointer + indirect access         |
| Add N numbers — auto increment (6M)         | Code + explain (R2)+ saves one instruction vs indirect        |
| Memory Mapped vs I/O Mapped (6M)            | 5-point comparison table                                      |
| Assembler directives (8M)                   | EQU, ORIGIN, DATAWORD, RESERVE, END, RETURN + example program |
| Format of assembly statement (4M)           | 4 fields: Label, Operation, Operands, Comment + example       |
| Two Pass Assembler (6M)                     | Pass 1: symbol table, Pass 2: code generation                 |

|                                      |                                                               |
|--------------------------------------|---------------------------------------------------------------|
| Stack PUSH and POP with diagram (8M) | Definition + SP role + PUSH code + POP code + Figure 2.14     |
| Safe PUSH and POP (6M)               | Boundary checking + SAFEPUSH + SAFEPOP code                   |
| Subroutine linkage (8M)              | CALL/RETURN mechanism + Link Register + Figure 2.16           |
| Subroutine nesting (6M)              | Problem with link register + stack solution + LIFO order      |
| Parameter passing (8M)               | Methods + example with LISTADD subroutine                     |
| Logical shifts (6M)                  | LShiftL, LShiftR, AShiftR with diagrams + BCD packing program |
| Rotate instructions (6M)             | All 4 types with diagrams + explanation                       |
| RPN (4M)                             | Infix/Prefix/Postfix + stack evaluation + example             |
| Stack vs Queue (6M)                  | Full comparison table + differences                           |
| EA calculation problems (6–8M)       | Know all 10 EA formulas + apply to given register values      |

---

**vtu-easy.com**

*Your complete VTU study companion — Simple, Clear, Exam-Ready*